Requirements

Image Processing Requirements

The image processing requirements are broken in to

Foundational Image Processing Requirements

ID	Type	Requirement Description
F.FOU-1	Functional	The software shall reliably segment ultrasound video frames to isolate the B-mode image, the Doppler graph, and the scale areas dynamically.
F.FOU-2	Functional	The system shall isolate the depth scale, the actual ultrasound image, and the placement vector from the B-mode region.
F.FOU-3	Functional	The system shall separate the green ECG trace from the white Doppler waveform within the Doppler region.
F.FOU-4	Functional	The system shall read the numerical values of the depth and velocity scales to obtain the scale of the images.
F.FOU-5	Functional	The system shall track the recording timeline.
F.FOU-6	Functional	The system shall accommodate specific ultrasound views where the Doppler waveform is entirely absent.
F.FOU-7	Functional	The system shall locate the on-screen bounding green "Z" box to establish the initial region of interest and utilize a breadth-first search to locate the surrounding vessel walls.
NF.FOU-1	Non-Functional	The software shall handle data interruptions.
NF.FOU-2	Non-Functional	The system shall maintain processing continuity if the depth or scale changes dynamically mid-recording.
NF.FOU-3	Non-Functional	The system shall accommodate recordings where the time scale changes dynamically during the video.
NF.FOU-4	Non-Functional	The system shall maintain tracking continuity when unexpected physical artifacts occur during the ultrasound recording, such as patient coughing, laughing, or hyperventilating.

Accurate Characterization of Vessel Geometry

ID	Type	Requirement Description
F.VES-1	Functional	The software shall accurately extract vessel diameter perpendicular to the vascular walls in longitudinal views.

F.VES-2	Functional	The system shall calculate the metrics for multiple vessel candidates in the B-mode view to determine the best candidate for tracking.
F.VES-3	Functional	The system shall determine the cross-sectional area of circular vessels in transverse views using edge detection.
F.VES-4	Functional	The system shall calculate the metrics for multiple vessel candidates in B-mode and transverse views to present the best candidate while having cached back-ups.

Extraction of Blood-Flow Information

ID	Type	Requirement Description
F.FLO-1	Functional	The system shall extract time-average blood velocity from the Doppler waveform.
F.FLO-2	Functional	The system shall calculate estimated volumetric blood flow using the extracted velocity and calculated diameter.

Test Harness Requirements

These requirements are ongoing. The data has not been explored enough to know exactly what the harness needs, so they will be refined as it is.

Purpose

A general-purpose harness for testing the functionality of the system on any ultrasound video, regardless of the procedure being performed (baseline, FMD, hypoxia, exercise, etc.). It does not need to know which procedure a video comes from.

Requirements

ID	Type	Requirement Description
F.TES-1	Functional	Accept any ultrasound video the system accepts as input.
F.TES-2	Functional	Test the system's functionality, from measurement extraction (diameter, CSA, velocity) to calculations (e.g. volumetric blood flow) and the other processing the system performs.
F.TES-3	Functional	Run the same tests on any video without being tuned to a specific one.
F.TES-4	Functional	Compare system output against ground truth where it exists.
F.TES-5	Functional	Report results in a form the team can review.

F.TES-6	Functional	Initial verification is on the FMD video.

To be defined

ID	Type	Requirement Description
TBD.TES-1	To Be Defined	Ground truth format.
TBD.TES-2	To Be Defined	Metrics and accuracy targets.
TBD.TES-3	To Be Defined	Full list of functions under test.

UI Requirements

Functional Requirements

ID	Type	Requirement Description
F.GUI-1	Functional	The system shall provide an interactive GUI enabling human review and manual override of automated edge-detection measurements.
F.GUI-2	Functional	The GUI shall project an augmented, high-contrast visual edge-detection spline over the original B-mode ultrasound video.
F.GUI-3	Functional	The GUI shall visually distinguish the primary detected vessel boundary while keeping alternative candidate boundaries selectable by the operator.
F.GUI-4	Functional	The GUI shall provide dedicated controls for operational states: Process, Edit, Pause, Cancel, and Undo.
F.GUI-5	Functional	The system shall log and preserve original machine calculations alongside human manual edits for full audit traceability.
F.GUI-6	Functional	The GUI shall display and allow interactive verification of the green "Z" bounding box and region of interest (ROI) placed around vessel walls.

Non-Functional Requirements

ID	Type	Requirement Description
NF.GUI-1	Non-Functional	Augmented edge-detection spline overlay shall be rendered with a system display latency < 100 ms.
NF.GUI-2	Non-Functional	The GUI shall remain fully responsive to user input and window scaling during active video processing without UI thread blocking.
NF.GUI-3	Non-Functional	The software GUI shall initialize into a ready state within 10 seconds on standard lab desktop workstations.
NF.GUI-4	Non-Functional	The interface shall use a high-contrast visual palette ensuring overlay lines remain legible across variable greyscale ultrasound

		backgrounds.
NF.GUI-5	Non-Functional	UI layout, controls, and video overlays shall render consistently across standard monitor resolutions and aspect ratios.
NF.GUI-6	Non-Functional	Active ROI parameters and session configurations shall be cached to prevent data loss upon accidental window minimization or closing.
NF.GUI-7	Non-Functional	The GUI shall display clear, non-technical error notifications when encountering corrupted files or unsupported video formats.

Output Requirements

PhysioMerge-Specific Outputs and Integration

ID	Type	Requirement Description
F.OUT-1	Functional	**PhysioMerge Integration:** Compatible with CSV exports that capture all extracted and calculated metrics.
F.OUT-2	Functional	**Vessel Diameter:** Extracted perpendicular to vascular walls in longitudinal views.
F.OUT-3	Functional	**Vessel Cross-Sectional Area:** Calculated for circular vessels in transverse views using edge detection.
F.OUT-4	Functional	**Time-Averaged Blood Velocity:** Extracted from the white Doppler waveform.
F.OUT-5	Functional	**Estimated Volumetric Blood Flow Rate:** Calculated using the extracted velocity and diameter (targeted in mL/min).
F.OUT-6	Functional	**Physical Pixel Scale/Calibration:** Derived from reading on-screen depth and velocity rulers (to within 1mm accuracy).
F.OUT-7	Functional	**Recording Timeline/Timestamp Tracking:** Time elapsed/processed across the Doppler and video timeline.
F.OUT-8	Functional	**ECG/Cardiac Signals:** Separated green ECG trace for cycle/diastole synchronization.
F.OUT-9	Functional	**Command Line Interface (CLI) Callable:** Integrate with software by calling it directly as CLI arguments for batch-processing pipelines.

GUI Outputs & Display Features

ID	Type	Requirement Description
F.OUT-10	Functional	**Augmented Video Overlay:** GUI displays an augmented, high-contrast visual edge-detection spline projected directly on top of the greyscale B-mode ultrasound video with low-latency and best vessel candidate detected from B-mode view.

F.OUT-11	Functional	**Track Revision History:** If researcher manually adjusts or edits an automated measurement, GUI preserves original automated machine result alongside the manual human edits to ensure traceability.
F.OUT-12	Functional	**Notification & Status:** Presents non-technical error notifications for unsupported formats or corrupted files and displays processing status/caching of user-defined ROI parameters to prevent data loss if window closes.
NF.OUT-1	Non-Functional	**Performance Benchmarks:** GUI must initialize into a ready state within reasonable timeframe on lab desktop hardware and remain responsive to window scaling and user inputs without freezing during heavy processing workloads.

Hardware & Environment Constraints

ID	Type	Requirement Description
C.ENV-1	Constraint	**Network:** Strictly Offline (System will never be hooked up to the internet).
C.ENV-2	Constraint	**Target Hardware:** NVIDIA rtxA5000 GPU and Xeon w-2245 CPU, 32 GB RAM.
C.ENV-3	Constraint	**Fallback Hardware:** Must support fallback for lower-end systems.
C.ENV-4	Constraint	**Language/Framework:** Python-based integration.

Testing

Test Dataset Design

The test dataset should include the following cases. *Views and Vessels*

ID	View	Vessel	Doppler	Tests
V1	Longitudinal	Common carotid artery (CCA)	Yes	Diameter, velocity, flow (main use case)
V2	Longitudinal	Brachial artery	Yes	FMD target vessel
V3	Transverse	CCA/Jugular vein	No	CSA of a round artery; handling recordings with no Doppler
V5	Transverse	Artery and vein both in view	Yes	Picking the right vessel; noisy Doppler

Protocols

ID	Protocol	Tests
P1	Baseline (resting)	Happy path
P2	FMD: baseline → 5 min cuff occlusion → release	Small diameter changes (a few tenths of a mm); fast flow spike after release; probe motion at cuff inflation/deflation
P3	Hypercapnia	Hyperventilation and neck movement
P4	Exercise (cycling, rowing)	Large motion; large velocity and heart-rate changes

Change of Settings

ID	Setting change	Tests
S1	Depth changed mid-recording	Re-reading the depth scale; new mm/px
S2	Velocity scale inverted (negative on top)	Reading scale sign
S3	Doppler baseline shifted	Reading scale offset
S4	Velocity scale set too low (aliasing)	Detecting peaks that wrap around the scale
S5	Sweep speed (time scale) changed	Re-calibrating time per pixel
S6	Doppler line/box moved or angle changed	Re-finding the ROI; angle-corrected diameter
S7	Steep insonation angle (> 60°)	Flagging unreliable velocities

Bad Data

ID	Event	Tests
A1	Cough, laugh, swallow, talking	Motion tracking and recovery
A2	Probe slips; vessel drifts across the frame	Tracking across frames
A3	Image goes off-screen or blank briefly	Detecting bad frames, then resuming
A4	Flicker	Robustness of edge detection
A5	Probe pressure collapses a vein	Distinguishing artery from vein
A6	Noisy or missing ECG trace	R-peak detection fallback
A7	Poor image quality (deep vessel, larger subject)	Low-contrast walls; worse mm/px

Danny Green Validation

The test plan should cover the main functional components below:

ID	Component	Description
DG-1	Acquisition & Calibration Module	Video control (rewind or fast-forward), 8-bit frame with ~16fps, pixel-to-millimetre scaling.

DG-2	ECG Signal Detection	Max/min detection algorithm to identify R waves for end-diastolic phase capture.
DG-3	Edge-Detection with Wall-Tracking	Binary thresholding, column-by-column contiguous zero counting and median filtering across 200–400 pixel columns (depending on frame).
DG-4	Post-Processing	Baseline averaging and 3rd order polynomial curve fitting for peak %FMD and %GTN responses.

Unit Tests

ID	Test	Test Description	Expected Result
UT-1	Calibration Transform	Input synthetic binary images with known pixel distances (i.e.: 10px, 50px, 100px, 200px).	Scale factor calculation $S = \delta{mm}/\delta{px}$ is accurate with no floating point drift.
UT-2	Binary Zero Counter	Feed test columns with alternating black/white patterns, fragmented lumen segments, near-wall noise, and clean lumens.	Correctly identifies the longest sequence of binary zeros without misidentifying false walls.
UT-3	Median Filter Robustness	Inject outlier columns into synthetic 200–400 column arrays.	Median calculation rejects outliers and spikes, maintaining the true median diameter.
UT-4	Polynomial Fit Test	Feed pre-calculated polynomial datasets (with known inflection and maximum points) into the 3rd-order curve fitting function.	Output matches the mathematical ground truth for peak dilation and time-to-peak.

Video Segmentation Method

Background

Technical Terms

- **FFmpeg**: open source software engine powering all digital video and audio processing
  - Provides massive suite of C libraries that know how to read, write, and compress most video/audio formats ever created
  - Decodes/encodes video data by converting compressed video data into raw pixel data (arrays) for screen display, and compressing back, to files to save space
- **Inter-frame compression**: modern video formats save one complete, full-quality image (aka Keyframe or I-frame) — next few dozen frames record the differences from the previous frame
  - Ex translation of pixels, saturation
- **Decode** (in frame processing): computational process of taking compressed instructions and performing math calculations to rebuild a full viewable grid of pixels
- **Demuxe**: split a video file into its separate video & audio tracks (muxe = combining)

- **YOLO** (You Only Look Once): deep learning model family for real-time object detection
  - Looks at entire image once, predicting bounding boxes & class labels
- **Optical Character Recognition (OCR)**: software that translates images of typed/handwritten text into machine-encoded text

Primary Requirements/Considerations:

1. **Ingest.mp4 files**: Accept .mp4 files as input and convert to frames for processing.
2. **Dynamic screen segmentation**: identify and segment different components captured in each frame, where each image may or may not contain a B-mode ultrasound image, the Doppler waveform, and ECG data, or some combination of two or three.
3. **B-mode outputs**: identify blood vessel structures in transverse or longitudinal views, take measurements like perpendicular diameter and cross-sectional area respectively.
4. **Doppler outputs**: envelope trace, handle inverted image, phase identification.

Ingesting MP4 Files

OpenCV (cv)

- Cost: free for commercial use
- Industry standard for this kind of work
- Best for: general computer vision, image processing, AI tasks
  - Relies on FFmpeg under the hood
- Output: NumPy arrays (BGR color format) — highly compatible with AI models & std math ops

PyAV (av)

- Cost: free
- Best for: high performance, precise frame seeking, direct FFmpeg control
  - Direct Pythonic bindings to FFmpeg's C libraries
  - Allows for exact timestamp seeking (w/out having to decode every prior frame)

MoviePy (moviepy)

- Cost: free
- Best for: video editing, composition, adding audio
- More for video manipulation than computer vision
- Slower than previous two

Decord

- Cost: free
- Best for: deep learning, machine learning pipelines, batching
  - Heavily optimized for slicing & batching video frames directly into GPU memory or PyTorch/TensorFlow tensors
  - Good for things like developing video-based neural network

Discussion

- OpenCV and PyAV are probably the main contenders
- OpenCV abstracts away complexities and outputs a NumPy array — straightforward for computer vision tasks
  - Allows for rapid prototyping - 2 lines of code to get a frame
  - Native language of YOLO & OpenCV filtering ecosystems
  - Inefficient for heavy I/O due to the layers of abstraction

    - Decodes sequentially → activities like frame-skipping (seeking) become expensive since it has to decode intermediate frames
    - Strips & ignores audio streams
- If we want to batch process, PyAV becomes the better choice
    - Provides direct, Pythonic bindings to FFmpeg's C libraries
    - Executes video demuxing & decoding with less CPU overhead
    - Steeper learning curve
    - Manually handle streams, packets, frames & explicitly convert output to standard NumPy if needed
- Ex if we had 500 ultrasound mp4 videos, PyAV offers:
    - Efficient frame seeking & downsampling (ie. if we only need 5 FPS instead of 30)
    - Multiprocessing: handles multithreading better, less constrained by Python's GIL during decoding phase — we can spin up multiple worker processes where each uses PyAV to decode a different mp4 in parallel
    - Access to audio stream (if future work needs the audio signal for Doppler shift)
- Can do grayscale only to save on processing memory & compute time
- Compare export FPS (30 or 60 typically) vs human heart beat (1 Hz/60 BPM) — could process every 2nd/3rd frame

Planning

- Not necessary to decide immediately — can implement an adapter pattern
    - Ex use a VideoLoader class, starting with one of them arbitrarily for MVP (probably OpenCV)
- Work on other components first — FFmpeg packet logic may be more complex and low priority as key features won't know the difference

Dynamic Screen Segmentation

Possible Combinations:

- B-mode only
- B-mode + Doppler
- B-mode + ECG only
- B-mode + Doppler + ECG
- Doppler only
- Doppler + ECG only
- (ECG only)
- Note: sidebar with measurements stays static (confirm)

Object Detection Approach

- Train a lightweight model (YOLOv11, YOLOv8-nano)
    - Tools like YOLO would require us to label a set of training frames with bounding boxes for B-Mode Window, Doppler Window, and ECG Window
    - Extract frames from different video recordings (avoid same video → will cause overfitting), enough for each window and layout
    - Upload frames to a computer vision platform
    - Create 3 class labels: b_mode, doppler, ecg
    - Draw bounding boxes tightly around exact pixel areas where data lives
- For each new video, run the YOLO model on the first frame & extract bounding box coordinates → now we have exact pixel crop regions to route to the subpipelines

- Setup cost: high - manually labelling hundreds of frames to train a model
- Hardware: requires a GPU for optimal speed

Temporal Variance Approach

- Temporal variance/background subtraction approach = math/stats based approach
- Process:
  - Take a stack of video frames
  - Calculate mathematical variance of each individual's pixel intensity across time
  - Draw bounding box around bright "blobs"
- Assumes background, UI borders, scales/text don't move → variance is exactly zero vs B-mode imaging of pulsating blood vessels and sweeping waveforms (doppler) constantly changing → higher variance
- Set up cost: zero — no training data required, works purely on math
- Struggles to handle opening/closing and changing in sizes of windows robustly

Assumption - Static Scale Lines

- Assuming that scales do not change and stay in their position per view combination, we only have a few main configurations:
  - B-mode only: b-mode scale on right edge (blue)
  - Doppler/ECG only or both: scale on left edge (white)
  - B mode + Doppler and/or ECG: B-mode scale on right edge, extending from top edge to halfway down the screen, Doppler/ECG scale on left edge, extending from middle to bottom edge
- Key features:
  - Doppler Y-axis has perfectly vertical stack of identical white dashes with numbers on the left
  - Doppler Y-axis labelled with "Velocity (cm/s)"
  - B-mode Y-axis = blue tick, aligned right
- Use *Regions of Interest*:
  - B mode ROI: tall, narrow rectangle on top right edge of the screen
    - Sum the nonblack pixels in the ROI, if passes a certain threshold, scale is there and B mode is active
  - Doppler ROI: tall, narrow rectangle near the bottom left edge of the screen
    - Can use "kernel"/filter out horizontal text, noise, speckles — leaving only the white vertical line
    - If vertical line is there, doppler is there
- Define bounding boxes relative to detected scales - Anchor Offset Architecture
  - Record exactly how many pixels the image sits from the scale in each mode & save as a static config map in code
  - Something like:

```
UI_OFFSETS = {
    "B_MODE_ONLY": {
        "b_image_x_start": anchor_x - 800,
        "b_image_x_end": anchor_x - 50,
        "b_image_y_start": 100,
        "b_image_y_end": 900
    },
    "DUPLEX_MODE": { # B-mode + Doppler
```

```
            "b_image_x_start": b_anchor_x - 500,
            "b_image_x_end": b_anchor_x - 50,
            "doppler_x_start": d_anchor_x + 20,
            "doppler_x_end": d_anchor_x + 900
        }
    }
```

B-mode Sub-pipeline

Pixel-to-Millimeter Calibration

- Use tick markers + depth measurement on sidebar
- Use OCR tool to read text - depth measurement will be the same as the number of units of measurement from top to bottom of image
- Can also use template matching/contour detecting to find distance between tick marks

Longitudinal Vessel Segmentation

- Vessel = dark tube between 2 lighter walls
- Segment lumen, calculate perpendicular vector to centre line of the vessel
- Measure distance between upper & lower boundary pixels

Transverse Vessel Segmentation

- Vessel = dark, pulsating circle
- Use image segmentation model to mask lumen (dark circle)
- Calculate area by counting mask pixels & multiplying your calibration factor

Doppler Sub-pipeline

Envelope Trace

- Tracing: outlining the outermost edge of the spectral display to track the peak or mean blood flow velocities over time
- Spectral display: the vertical axis represents velocity (or frequency shift) and the horizontal axis represents time
- Envelope: bright outer edge of spectral waveform shows the highest (peak) or average velocity of red blood cells moving through the sample volume at any given moment.
- Extract max velocity over time (outer edge of white waveform)
    - Column wise maximum intensity projection (find highest bright pixel in each vertical colum) & smooth with a Savitzky-Golay filter (tool available in scipy)

Phase Identification

- Scipy has signal.find_peaks — can be used on smoothed envelope trace
- Highest peak = peak systolic velocity
- Lower point (valley) immediately preceding a systolic peak = end diastolic velocity

Inverted Waveforms

- Use OCR on the y-axis scale
- If the numbers above the zero have a negative sign, the axis inverted
- Can use tools like cv2.flip() if needed

PhysioMerge Recon

Data Handling

- PhysioMerge (PM) expects a CSV with metadata at the top and the biometric data below. The exact format is not yet defined.
- PM automatically cleans messy signals by deleting or filtering them.
- PM records whether data was deleted as a true/false flag in the file metadata, and keeps a log of what was deleted so it can be reviewed later and is never lost. We receive this field.

Available Commands

The commands we can use, based on what is established:

Capability	Purpose for us
Morphology calculations	Per cardiac cycle, so we do not redo them
Measurements	Mean, systole and diastole
Data deletion / filtering	Handling messy signals
`check` command	Error correction

Command names and syntax beyond `check` are not yet known.

New commands are not expected to be needed, since PM already covers what we need. If one is needed, we contribute it directly to PM so everyone benefits.

Additional Features Enabled

PM already provides morphology calculations per cardiac cycle, which overlaps with the morphological-data extension for Doppler waveforms. No other additional features have been identified.

Not Yet Known

- Command names and syntax
- Exact CSV format
- How PM invokes us beyond the CLI example

PhysioMerge Integration Spec

Architecture

The current pipeline is `Video -> Danny Green -> CSV -> PM`. Our software replaces Danny Green, so PM calls us directly.

Interaction Model

- Our software is a standalone service that PM calls (e.g. via CLI). It is not called as a package or imported by PM. That said, we may make it a callable package in the future, with CLI as fallback.
- We act as a black box: PM sends input, we return output. Nothing in the interface is specific to PM, so other systems could use us the same way.
- Our application is not concerned with where the video comes from.

Input

- Ultrasound video, provided by the caller.

Output

- A CSV with metadata at the top and the biometric data below, which is what PM expects.
- Exact format not yet defined.

Division of Responsibility

- **Us:** analysis of the video.
- **PM:** calculations it already provides (morphology per cardiac cycle, mean/systole/diastole measurements), data deletion/filtering and error correction via `check`.
- The final split is decided in the integration design.

Data Deletion and Revision

- PM cleans messy data automatically and records whether data was deleted as a true/false flag in the file metadata. We receive this field.
- PM keeps the log of what was deleted, so it is never lost. We do not log deletions ourselves.
- Manual human edits keep the original machine result alongside the revision.

Error Handling

- Error correction on messy data is done through PM's `check` command.
- Error handling needed on our end is not yet defined.

Extending PM

If a new command is needed, it is contributed directly to PM. This is not expected.

Open Items

- Exact CSV format
- Command names and syntax
- Invocation details beyond CLI
- Error handling on our end